POPPY: Scalable and Secure Spectral Centrality for Distributed Graphs via Homomorphic Encryption

Claire Guichemerre[1] Tristan Allard[1] Sofiane Azogagh[2] Sébastien Gambs[2]
Marc-Olivier Killijian[2] Amr El Abbadi[3]

[1]Université de Rennes, CNRS, IRISA

[2]Université du Québéc à Montréal

[3]University of California Santa Barbara

21 Nov 2025
Réunion MCGA, Montréal











Presentation Overview

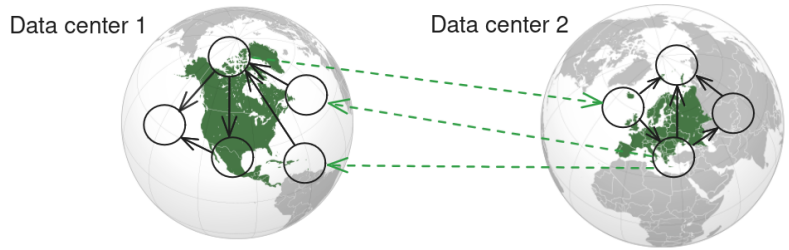
1. Introduction
2. Preliminaries
3. POPPY
4. Experimental results
5. Conclusion

Progress of the Talk

1. Introduction
2. Preliminaries
3. POPPY
4. Experimental results
5. Conclusion

Context

Geo-distributed data centers



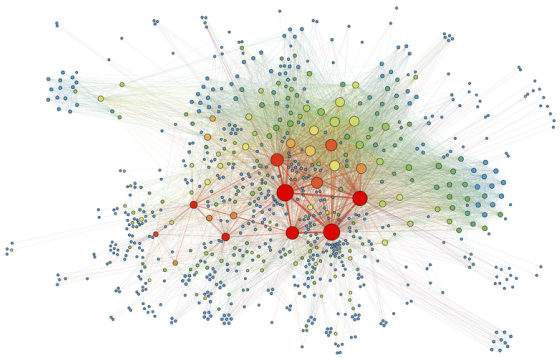
- Large graph data: common nowadays.
- For example graphs representing:
  - Large scale social networks (e.g., Facebook, Twitter (X), WhatsApp)
  - Transactions between bank accounts
- With personal data and partitionned in multiple data centers.

Goal

Identifying influential nodes in graphs is important!
For example:

- Preventing fake news
- Detecting financial fraud
- Accelerating information propagation

$\Rightarrow$ Focus in this work: **spectral centrality measures**.

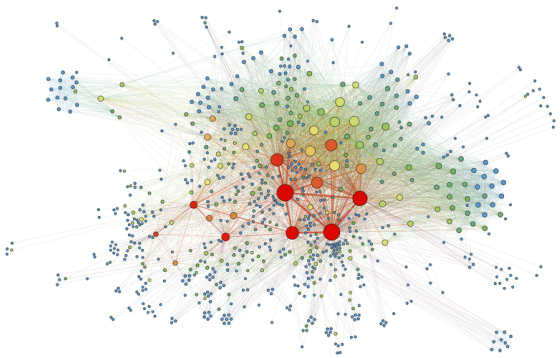


Goal

Identifying influential nodes in graphs is important!
For example:

- Preventing fake news
- Detecting financial fraud
- Accelerating information propagation

$\Rightarrow$ Focus in this work: **spectral centrality measures**.



But centralizing the graph is not an option:

- Laws might hamper personal data **transfer**: (e.g.,EU GDPR).
- Data centers might be mutually distrustful (e.g., distinct financial institutions).

POPPY: Our Solution

- Fully homomorphic encryption (FHE):
  - Allow *arbitrary* computations over encrypted data.
  - Continuous improvements during the last 15 years.

POPPY: Our Solution

- Fully homomorphic encryption (FHE):
  - Allow *arbitrary* computations over encrypted data.
  - Continuous improvements during the last 15 years.

Our goal: Use it for geo-distributed computation of spectral centrality measures!

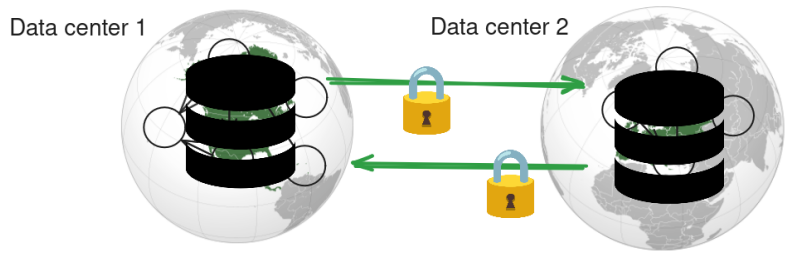


Figure: Secure information transfer between data centers.

Progress of the Talk

1. Introduction
2. Preliminaries
3. POPPY
4. Experimental results
5. Conclusion

Spectral centrality measures

- Assigns to each vertex a **real value** that quantifies the importance of the vertex.

Spectral centrality measures

- Assigns to each vertex a **real value** that quantifies the importance of the vertex.
- Based on the **eigenvector of the adjacency matrix** of the graph.

Spectral centrality measures

- Assigns to each vertex a **real value** that quantifies the importance of the vertex.
- Based on the **eigenvector of the adjacency matrix** of the graph.
- Spectral centrality measures are similar:
  - iterative,
  - based on the adjacency matrix,
  - and similar basic operations.

Spectral centrality measures

- Assigns to each vertex a **real value** that quantifies the importance of the vertex.
- Based on the **eigenvector of the adjacency matrix** of the graph.
- Spectral centrality measures are similar:
  - iterative,
  - based on the adjacency matrix,
  - and similar basic operations.
- Focus on the **Pagerank**:
  - $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ is a graph, $|\mathcal{V}| = n$.

Spectral centrality measures

- Assigns to each vertex a **real value** that quantifies the importance of the vertex.
- Based on the **eigenvector of the adjacency matrix** of the graph.
- Spectral centrality measures are similar:
  - iterative,
  - based on the adjacency matrix,
  - and similar basic operations.
- Focus on the **Pagerank**:
  - $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ is a graph, $|\mathcal{V}| = n$.
  - The pagerank (PR) of node $u \in \mathcal{V}$ at the t-th iteration is:

$$PR_t(u) = \frac{1-d}{k_{max}} + d \sum_{v \in E_u} \frac{PR_{t-1}(v)}{\deg_{out}(v)}$$

in which d is the dumping factor (typically 0.85) and k_{max} is the maximum in-degree across all nodes.

Spectral centrality measures

- Assigns to each vertex a **real value** that quantifies the importance of the vertex.
- Based on the **eigenvector of the adjacency matrix** of the graph.
- Spectral centrality measures are similar:
  - iterative,
  - based on the adjacency matrix,
  - and similar basic operations.
- Focus on the **Pagerank**:
  - $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ is a graph, $|\mathcal{V}| = n$.
  - The pagerank (PR) of node $u \in \mathcal{V}$ at the t-th iteration is:

$$PR_t(u) = \frac{1-d}{k_{max}} + d \sum_{v \in E_u} \frac{PR_{t-1}(v)}{\deg_{out}(v)}$$

    in which d is the dumping factor (typically 0.85) and k_{max} is the maximum in-degree across all nodes.
  - Initialization: $PR_0(u) = 1/n$ for all $u \in \mathcal{V}$.

Fully homomorphic encryption building block

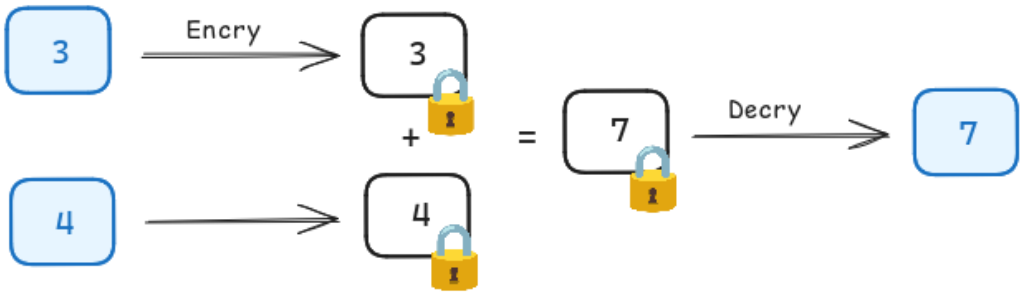


Figure: Addition over encrypted datas.

A fully homomorphic encryption (FHE) scheme allows us to

- Compute operations on ciphertexts without having to decrypt them beforehand (additions, multiplications).

Fully homomorphic encryption building block

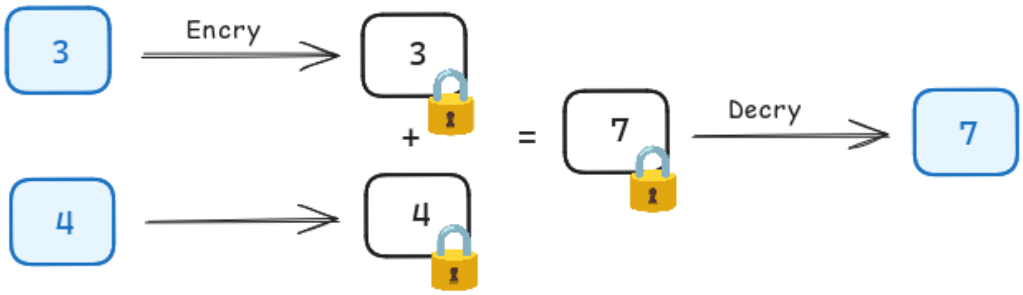


Figure: Addition over encrypted datas.

A fully homomorphic encryption (FHE) scheme allows us to

- Compute operations on ciphertexts without having to decrypt them beforehand (additions, multiplications).
- Perform a bounded number of **multiplications** on the **same** ciphertext. This number is the *multiplicative depth*.

Fully homomorphic encryption building block

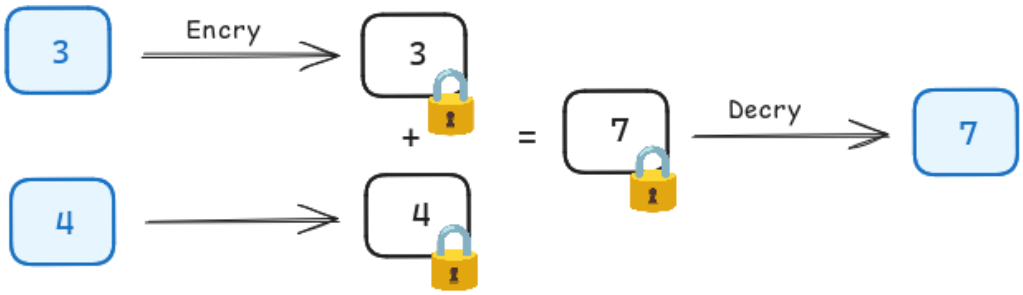


Figure: Addition over encrypted datas.

A fully homomorphic encryption (FHE) scheme allows us to

- Compute operations on ciphertexts without having to decrypt them beforehand (additions, multiplications).
- Perform a bounded number of **multiplications** on the **same** ciphertext. This number is the *multiplicative depth*.
- Possibility to perform more multiplications with a technic calls bootstrapping.

Fully homomorphic encryption building block

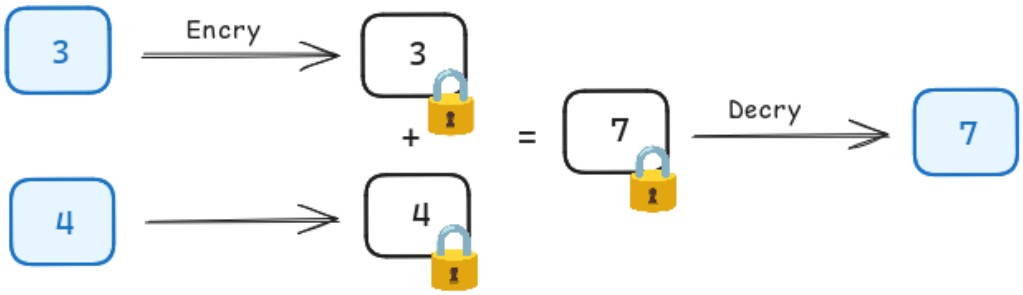


Figure: Addition over encrypted datas.

A fully homomorphic encryption (FHE) scheme allows us to

- Compute operations on ciphertexts without having to decrypt them beforehand (additions, multiplications).
- Perform a bounded number of **multiplications** on the **same** ciphertext. This number is the *multiplicative depth*.
- Possibility to perform more multiplications with a technic calls bootstrapping.
- The homomorphic operations are **costly**.

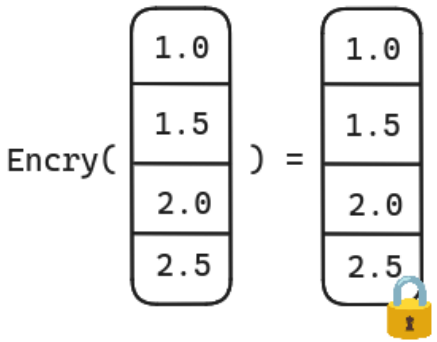
CKKS scheme

CKKS (Cheon Kim Kim Song [4]) performs operations over encrypted real data:

CKKS scheme

CKKS (Cheon Kim Kim Song [4]) performs operations over encrypted real data:

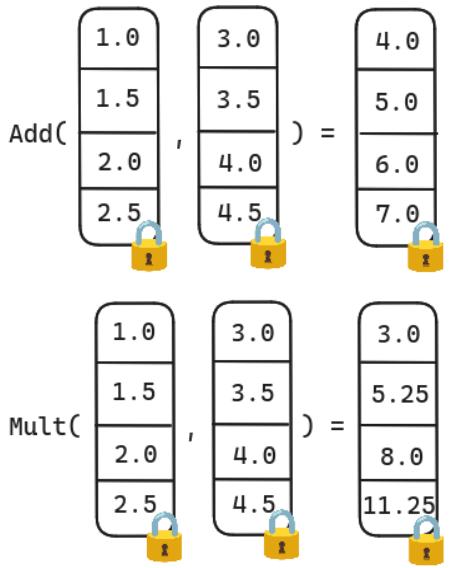
- A single ciphertext stores **multiple real numbers** (SIMD): data can be represented as a fixed-size **vector**.



CKKS scheme

CKKS (Cheon Kim Kim Song [4]) performs operations over encrypted real data:

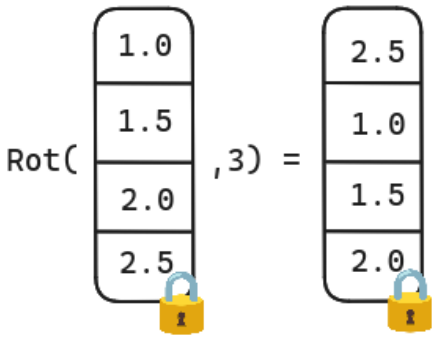
- A single ciphertext stores **multiple real numbers** (SIMD): data can be represented as a fixed-size **vector**.
- Additions and multiplications are **cell-wise** (SIMD).



CKKS scheme

CKKS (Cheon Kim Kim Song [4]) performs operations over encrypted real data:

- A single ciphertext stores **multiple real numbers** (SIMD): data can be represented as a fixed-size **vector**.
- Additions and multiplications are **cell-wise** (SIMD).
- An additional operation called **rotation** shifts the values in the vector (modular shift).

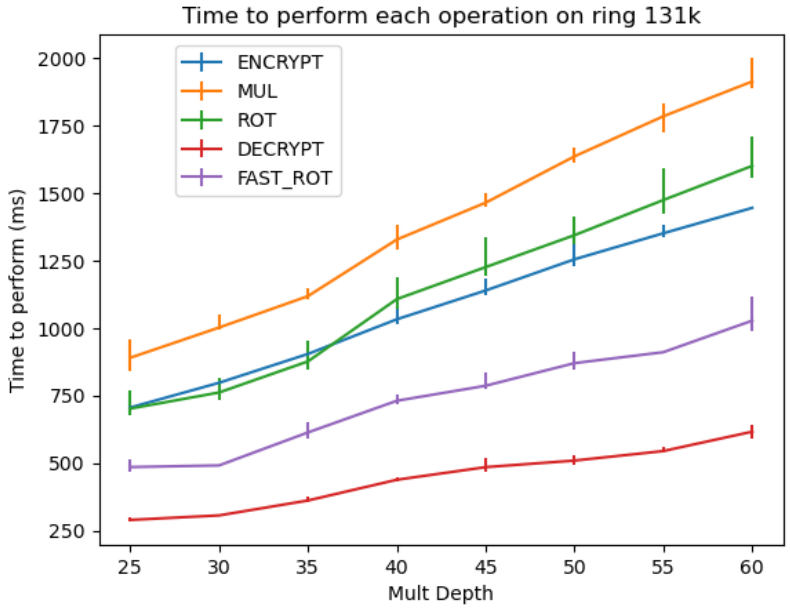


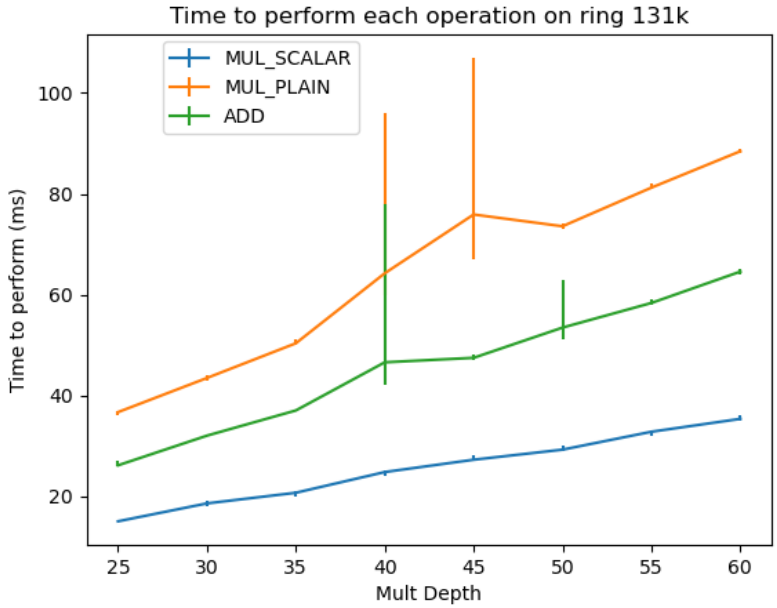
CKKS scheme

CKKS (Cheon Kim Kim Song [4]) performs operations over encrypted real data:

- A single ciphertext stores **multiple real numbers** (SIMD): data can be represented as a fixed-size **vector**.
- Additions and multiplications are **cell-wise** (SIMD).
- An additional operation called **rotation** shifts the values in the vector (modular shift).
- We can choose a specific **multiplicative depth** depending on our need:
  - Impact on the size of the vectors
  - Impact on the computation time

CKKS operations cost 1/2





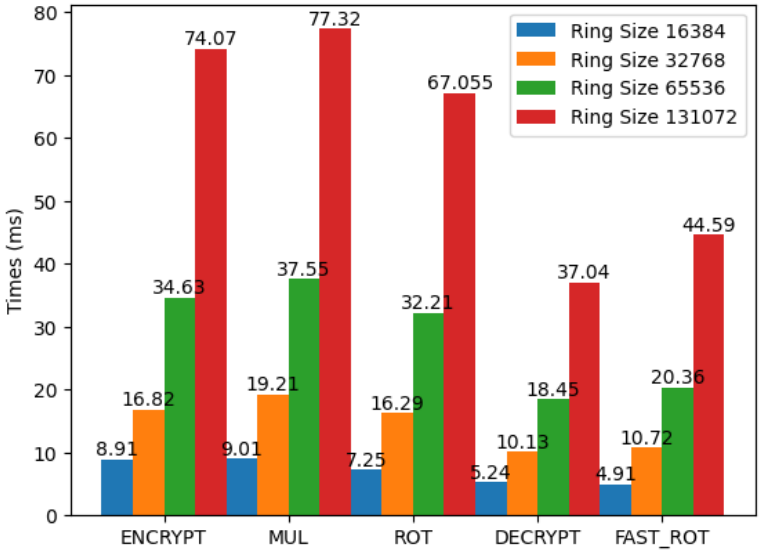
Specifications:

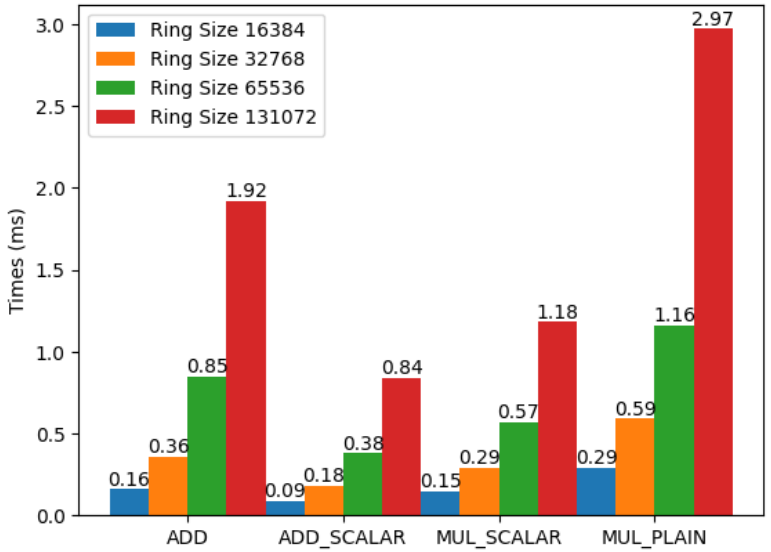
- 13th Gen Intel(R) Core i7-13850HX
- 27 cores
- 15GB RAM
- 5x558GB HDD

Graphs:

- Average, min and max over 10 repeatitions of the same operation
- Ring dimension fixed to 131072
- Open source library: OpenFHE

CKKS operations cost 2/2





Specifications:

- 13th Gen Intel(R) Core i7-13850HX
- 27 cores
- 15GB RAM
- 5x558GB HDD

Graphs:

- Average time over 100 repeatitions of the same operation with a multiplicative depth fixed to 4
- Open source library: OpenFHE

Related work



Privacy-preserving centrality measures in distributed graphs:

- **Closest work to ours:** [17] from Sangers et al., a privacy-preserving PageRank algorithm using the Damgård-Jurik additively homomorphic encryption scheme [5], an extension of the Paillier cryptosystem.

$\Rightarrow$ Vertex-centric computation paradigm similar to our baseline resulting in communication overhead.

Related work



Privacy-preserving centrality measures in distributed graphs:

- **Closest work to ours:** [17] from Sangers et al., a privacy-preserving PageRank algorithm using the Damgård-Jurik additively homomorphic encryption scheme [5], an extension of the Paillier cryptosystem.

$\Rightarrow$ Vertex-centric computation paradigm similar to our baseline resulting in communication overhead.

- Data analytics on graphs using **perturbation**: PGPregel [18]
- Centrality calculation on graphs using **SMPC** algorithms [8, 11, 12, 13]

$\Rightarrow$ Either loss of precision [18] or a different context [8, 11, 12, 13].

Related work

- Privacy-preserving centrality measures in distributed graphs:
  - **Closest work to ours:** [17] from Sangers et al., a privacy-preserving PageRank algorithm using the Damgård-Jurik additively homomorphic encryption scheme [5], an extension of the Paillier cryptosystem.

  ⇒ Vertex-centric computation paradigm similar to our baseline resulting in communication overhead.
  - Data analytics on graphs using **perturbation**: PGPregel [18]
  - Centrality calculation on graphs using **SMPC** algorithms [8, 11, 12, 13]

⇒ Either loss of precision [18] or a different context [8, 11, 12, 13].

- Machine learning algorithm apply on encrypted graphs:
  - Graph encryption with **homomorphic encryption** [3, 14]
  - Link prediction over a distributed graph based on **Diffie-Hellman secret sharing** [6]: Crypto'Graph [2]

- Matrix factorization for recommendation using homomorphic encryption [10]

⇒ These works do not support computation of **centrality measures** and/or do not consider **distributed graphs over several data centers**.

Progress of the Talk

1. Introduction
2. Preliminaries
3. POPPY
4. Experimental results
5. Conclusion

Overview of POPPY

Three POPPY algorithms with differents trade-offs:

- A baseline which follows the "Think Like a Vertex" paradigm [15].
- POPPY$_{u_j}$ version using the SIMD capabilities of CKKS.
- POPPY$_M$ and POPPY$_{M2}$ versions with a matrix representation.

Overview of POPPY

Three POPPY algorithms with differents trade-offs:

- A baseline which follows the "Think Like a Vertex" paradigm [15].
- POPPY$_{u_j}$ version using the SIMD capabilities of CKKS.
- POPPY$_M$ and POPPY$_{M2}$ versions with a matrix representation.

We also have:

- A cost model for the three versions.
- Proofs of security and correctness.

Overview of POPPY

Three POPPY algorithms with differents trade-offs:

- A baseline which follows the "Think Like a Vertex" paradigm [15].
- POPPY$_{u_j}$ version using the SIMD capabilities of CKKS.
- POPPY$_M$ and POPPY$_{M2}$ versions with a matrix representation.

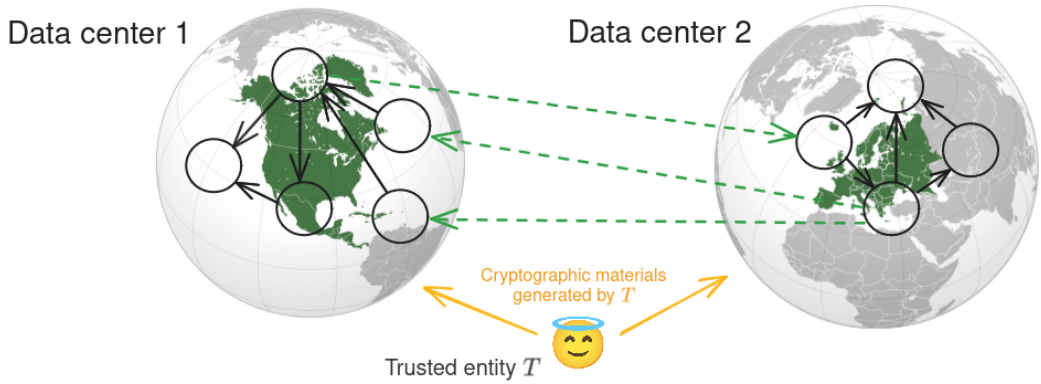
We also have:

- A cost model for the three versions.
- Proofs of security and correctness.

Extensions to cope with larger graphs:

- Exact pruning rules.
- Algorithms for overcoming the limited capabilities of the CKKS vectors.

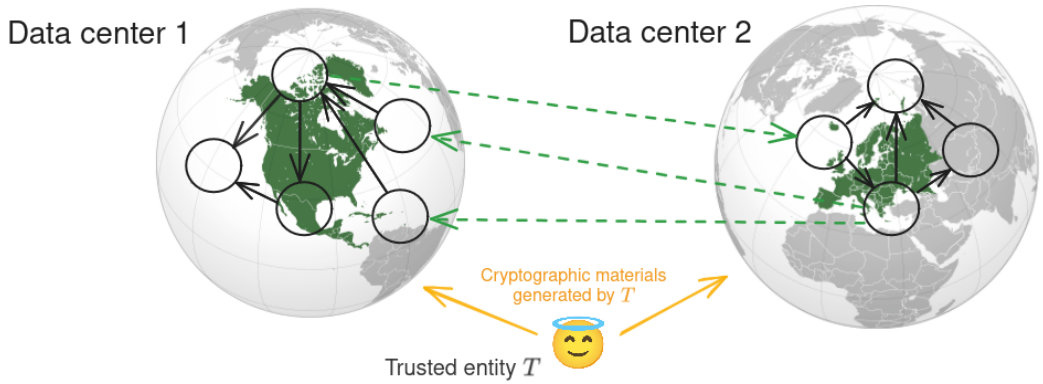
Participants



A **trusted entity** generates the cryptographic material.

- She shares the public key: each data center is able to encrypt.
- She keeps private the secret key: no data center can decrypt.

Participants



A **trusted entity** generates the cryptographic material.

- She shares the public key: each data center is able to encrypt.
- She keeps private the secret key: no data center can decrypt.

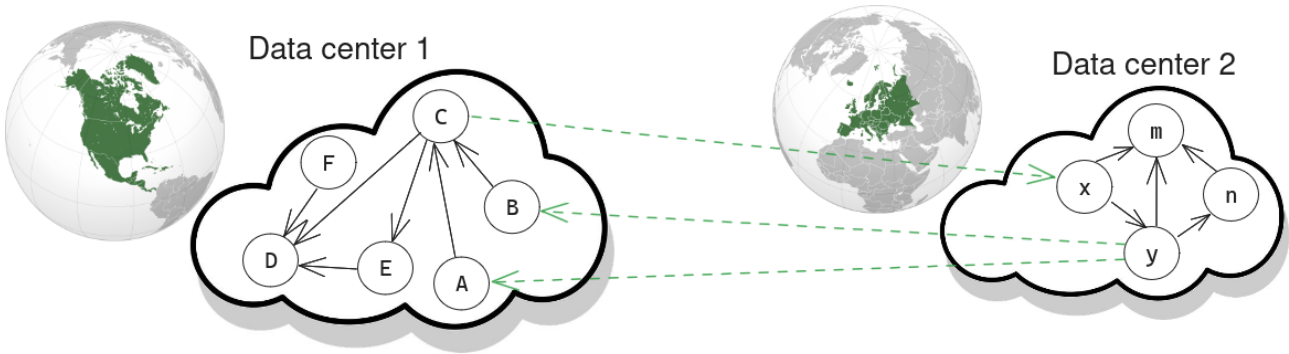
The data centers are **honest-but-curious**.

The baseline

- Naive approach which follows the "Think Like a Vertex" paradigm [15].

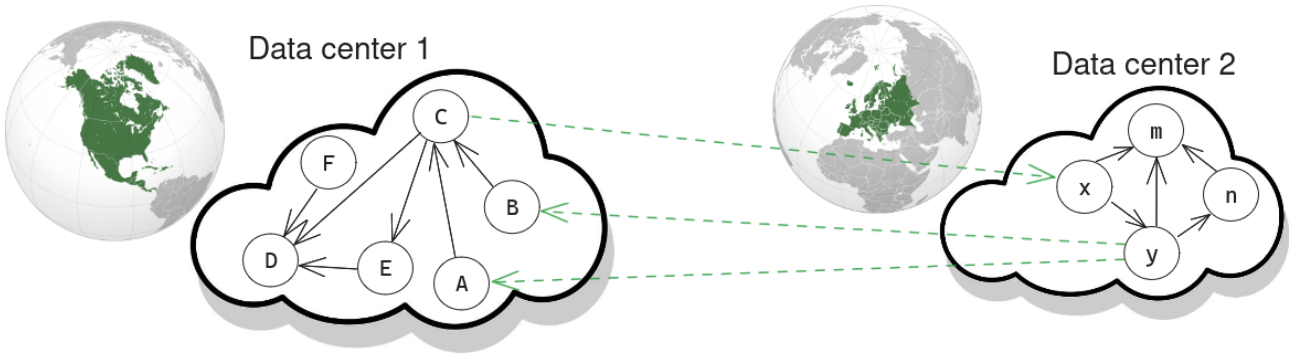
The baseline

- Naive approach which follows the "Think Like a Vertex" paradigm [15].
- Running example:



The baseline

- Naive approach which follows the "Think Like a Vertex" paradigm [15].
- Running example:

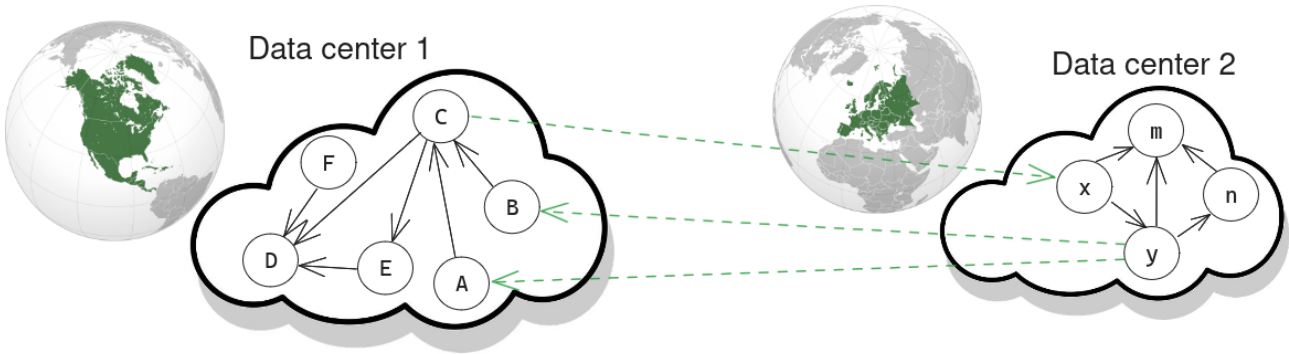


- Directly translates the cleartext algorithm into an encrypted form.
  - Each node u computes **iteratively** the algorithm:

$$\text{Encry}(PR_t(u)) = \frac{1-d}{k_{max}} + d \sum_{v \in E_u} Encry(\frac{PR_{t-1}(v)}{\deg_{out}(v)})$$

The baseline

- Naive approach which follows the "Think Like a Vertex" paradigm [15].
- Running example:

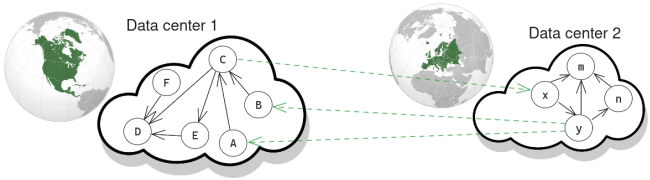


- Directly translates the cleartext algorithm into an encrypted form.
  - Each node u computes **iteratively** the algorithm:

$$\text{Encry}(PR_t(u)) = \frac{1-d}{k_{max}} + d \sum_{v \in E_u} Encry(\frac{PR_{t-1}(v)}{\deg_{out}(v)})$$

- Number of operations: linear in the number of nodes of the graph.

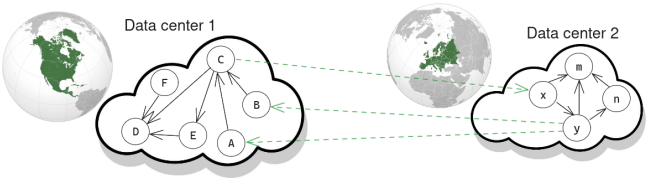
The limitations of the naive version



The baseline is not scalable:

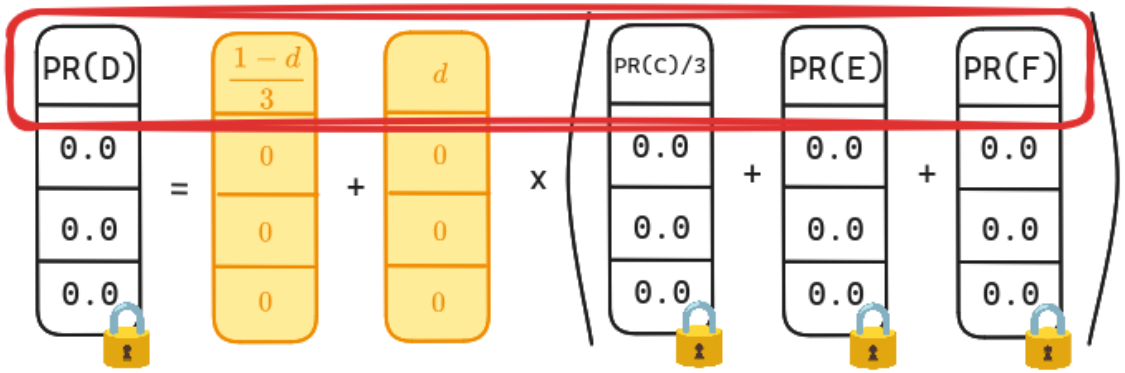
- Each node is a computing node.

The limitations of the naive version

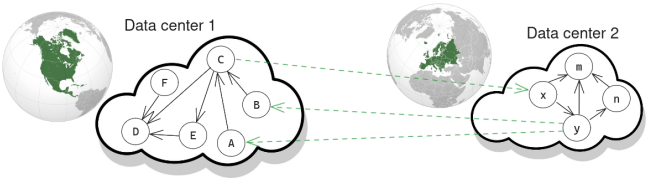


The baseline is not scalable:

- Each node is a computing node.
- The vectors are not well used: lots of zeros.

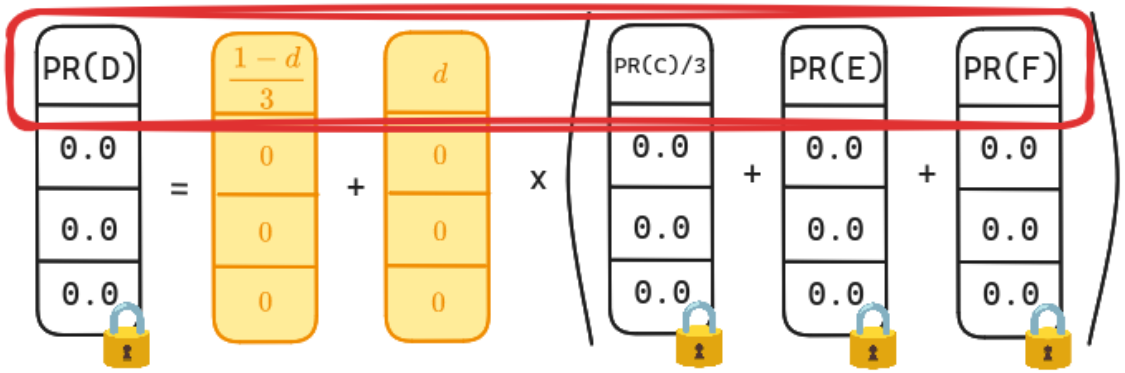


The limitations of the naive version



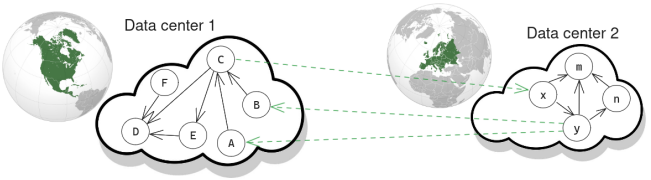
The baseline is not scalable:

- Each node is a computing node.
- The vectors are not well used: lots of zeros.



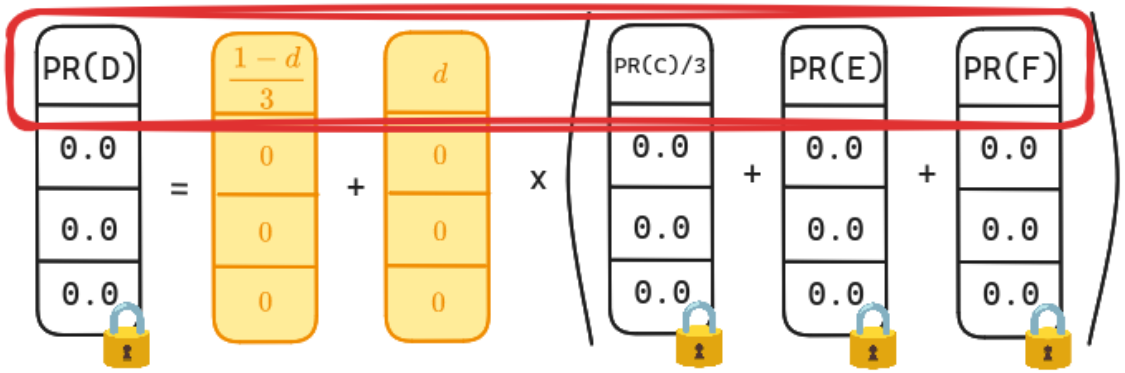
- Lots of useless operations.

The limitations of the naive version



The baseline is not scalable:

- Each node is a computing node.
- The vectors are not well used: lots of zeros.



- Lots of useless operations.
- Communication overhead.

POPPY$_{u_j}$

New approach: the POPPY$_{u_j}$ algorithm with new features

- Use the SIMD capabilities of CKKS

POPPY$_{u_j}$

New approach: the POPPY$_{u_j}$ algorithm with new features

- Use the SIMD capabilities of CKKS
- Divided in three main steps:
  - the *pre-processing step*
  - the *computation step*
  - the *completion step*

POPPY$_{u_j}$

New approach: the POPPY$_{u_j}$ algorithm with new features

- Use the SIMD capabilities of CKKS
- Divided in three main steps:
  - the *pre-processing step*
  - the *computation step*
  - the *completion step*

Intuition of POPPY$_{u_j}$

A **unique ciphertext** containing pagerank values of a data center in each slot.

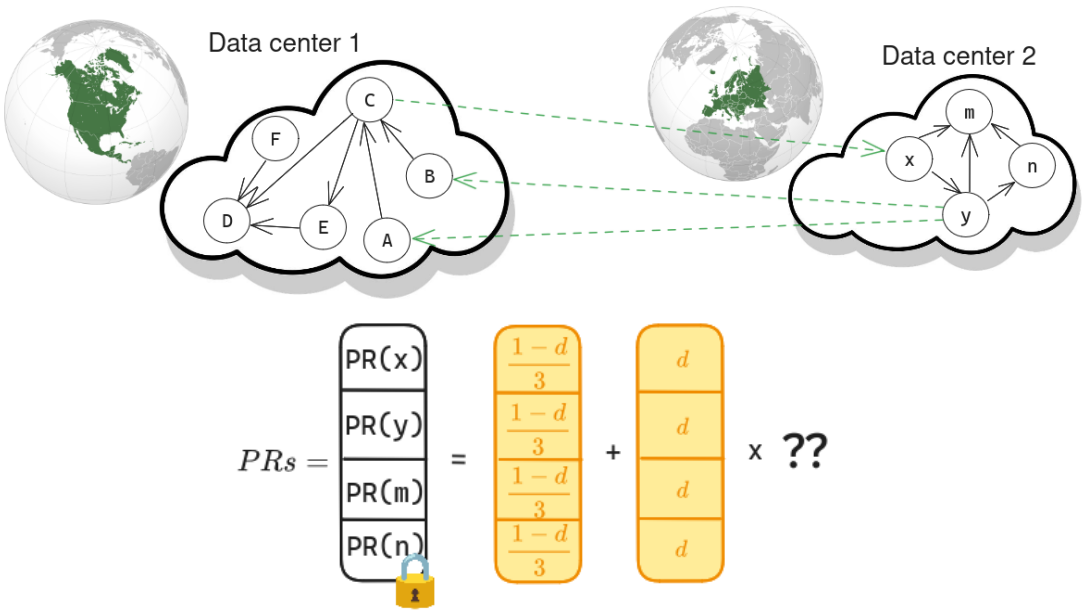
POPPY$_{u_j}$

New approach: the POPPY$_{u_j}$ algorithm with new features

- Use the SIMD capabilities of CKKS
- Divided in three main steps:
  - the *pre-processing step*
  - the *computation step*
  - the *completion step*

Intuition of POPPY$_{u_j}$

A **unique ciphertext** containing pagerank values of a data center in each slot.



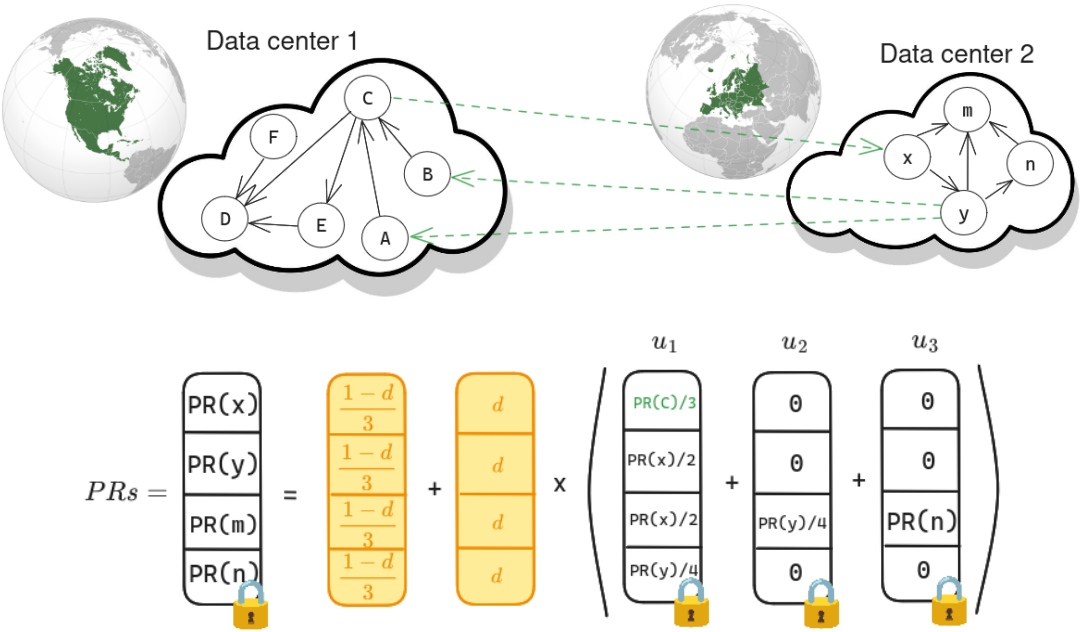
POPPY$_{u_j}$

New approach: the POPPY$_{u_j}$ algorithm with new features

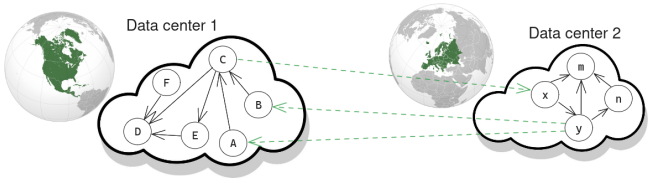
- Use the SIMD capabilities of CKKS
- Divided in three main steps:
  - the *pre-processing step*
  - the *computation step*
  - the *completion step*

Intuition of POPPY$_{u_j}$

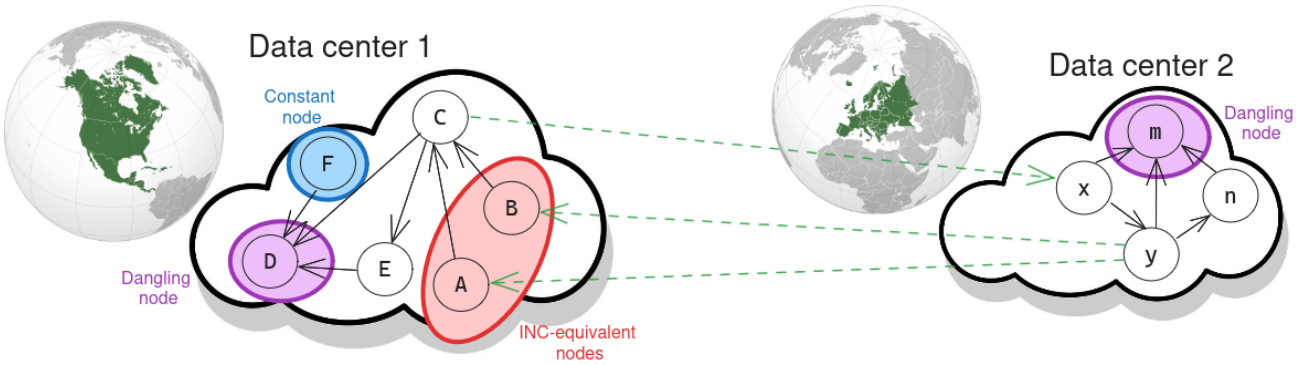
A **unique ciphertext** containing pagerank values of a data center in each slot.



Pre-processing step

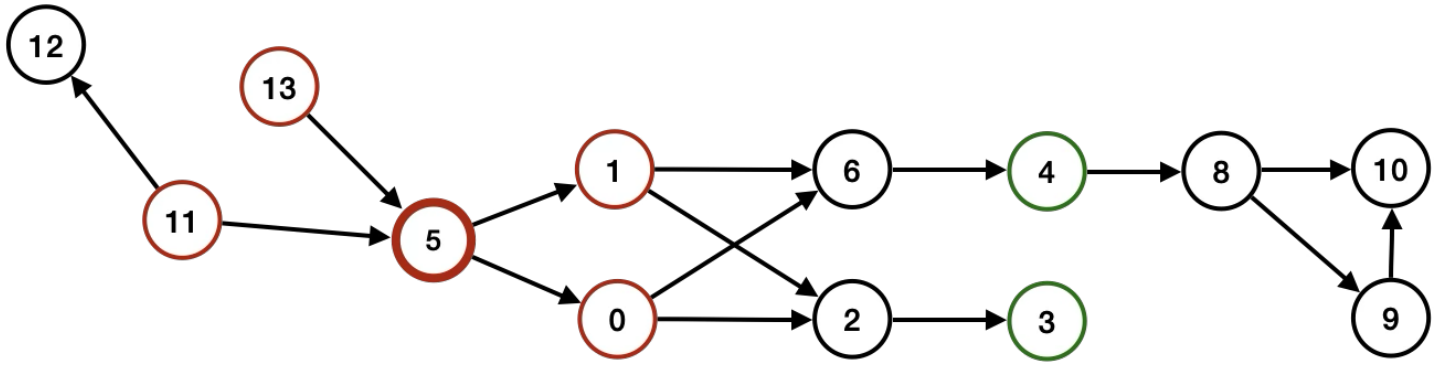


- Apply the pruning rules:
  - **Constant nodes:** nodes without incoming edges have a constant pagerank.
  - **Dangling nodes:** nodes without outgoing edges have no impact [9].
  - **Equivalent nodes:** two nodes that are part of the same *INC-equivalente class* have the same pagerank value.



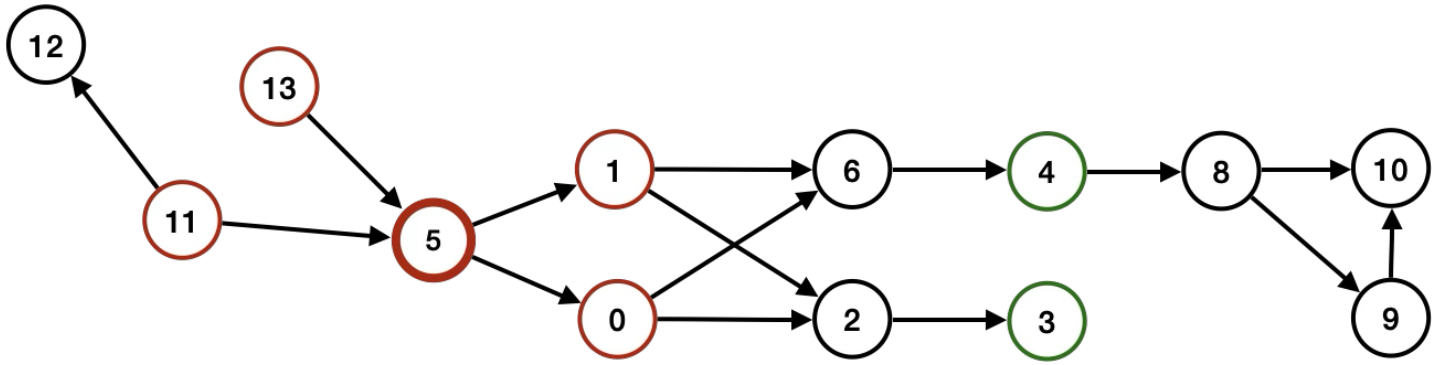
INC-equivalence intuition

- Generalization of orbits



INC-equivalence intuition

- Generalization of orbits



- We join nodes with the same pattern

Definition (Pattern of node t)

The pattern of node t is defined as the set of pairs $(s, w) \in (\mathcal{V}x\mathbb{R})$ for each edge $(s, t) \in \mathcal{E}$ where

$$w = \sum_{v \in \mathcal{C}(s)} \frac{1}{deg_{out}(v)}$$

and $\mathcal{C}(s)$ represents the set of nodes in the group of s. If t is a constant node (i.e., $\forall u \in \mathcal{V}, (u, t) \notin \mathcal{E}$), its pattern is $\emptyset$.

INC-equivalence algorithm

Input: A graph $\mathcal{G}$
Output: The pruned graph $\mathcal{G}_p = (\mathcal{V}_p, \mathcal{E}_p)$
$\mathcal{G}_p \leftarrow \mathcal{G}$
repeat
    $c \leftarrow$ True
    Let P be the pattern vector for each node of $\mathcal{G}_p$
    // Regroup nodes sharing similar pattern
    Let G be the vector of groups
    for each (u, v) in $(\mathcal{V}_p)^2$ s.t. $P[u] = P[v]$ and $u < v$ **do**
        $G[u]$.append(v)
    end for
    // Merge groups with more than one node
    for each group g in G with $|g| > 1$ **do**
        $c \leftarrow$ False
        Let r be the representative node of g
        for each $u \in g \setminus \{r\}$ **do**
            Replace all occurrences of u by r in P
            for each $x \in V_p$ s.t. $(u, x) \in \mathcal{E}_p$ **do**
                // Update the weights of x
                $P[x].weight \leftarrow P[x].weight + \frac{1}{deg_{out}(u)}$
            end for
        end for
    end for
    Update $\mathcal{G}_p$ by removing non-representative nodes in $\mathcal{G}_p$
until c
return $\mathcal{G}_p$

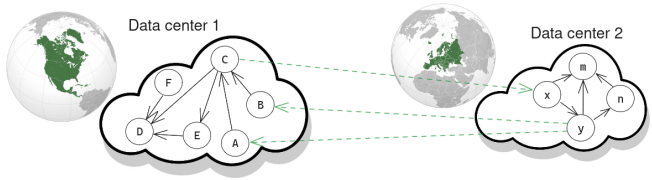
Comparison of Algorithm EQUIVALENCE with nauty and Traces [16]

Graph	$\#C$	$\#D$	Exclu. Pruning		Pruning Rate		Time (s)	
			[16]	Ours	[16]	Ours	[16]	Ours
Students	17	22	0	76	7.5%	15.1%	0.0003	0.003
Email-Eu-core	14	137	0	22	2.88%	5.07%	0.02	0.07
WikiVote	4734	1005	0	2956	18.4%	66.9%	35.4	0.34
P2P Gnutella	20	5941	0	816	6.69%	14.21%	18.6	0.50
Slashdot	0	44	449	184	18.89%	18.54%	6626	4.39
1000-nodes	746	113	0	339	43.6%	83.7%	0.38	0.01
5000-nodes	3819	527	0	1545	50.7%	87.4%	15.9	0.03
10000-nodes	7570	1104	1	3474	47.25%	87.2%	75.82	0.08

Graph	$\#C$	$\#D$	Memory (MB)		Distributed graph		
			[16]	Ours	$\#C$	$\#D$	Pruning Rate
Students	17	22	4.9	1.4	16	22	0.54%
Email-Eu-core	14	137	13.3	8.5	14	135	1.89%
WikiVote	4734	1005	39.7	22.9	371	909	18.07%
P2P Gnutella	20	5941	22.2	13.3	20	5520	4.95%
Slashdot	0	44	252	205.3	0	15	4.53%
1000-nodes	746	113	5.7	2.03	505	109	24.9%
5000-nodes	3819	527	11.3	4.2	2545	518	26.84%
10000-nodes	7570	1104	20.8	7.06	5120	1072	26.1%

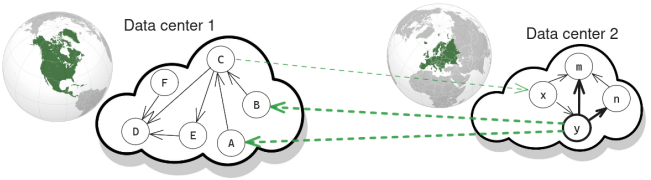
$\#C$ = constant nodes and $\#D$ = dangling nodes.

Pre-processing step

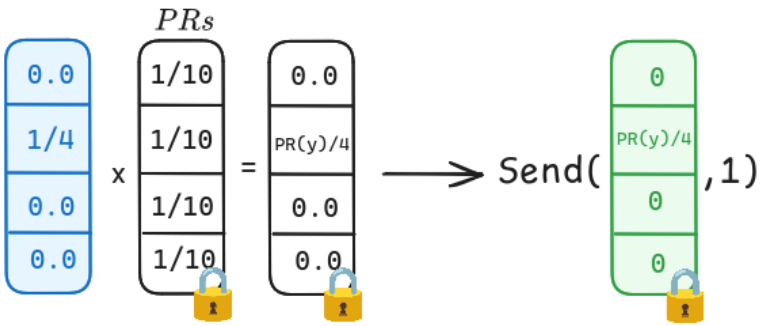


- Apply the pruning rules
  $\Rightarrow$ **Active nodes** are:
  - one representative node per `INC`-equivalente class.
  - nodes that are not constant.
  - nodes that are not dangling.

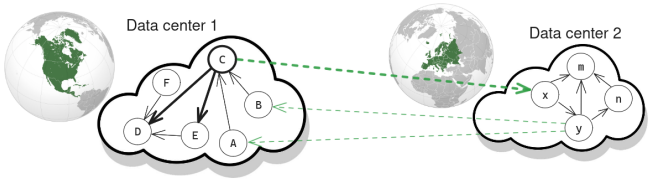
Pre-processing step



- Apply the pruning rules
  $\Rightarrow$ **Active nodes** are:
  - one representative node per `INC`-equivalente class.
  - nodes that are not constant.
  - nodes that are not dangling.
- Preparation for the first iteration: data center 2 masks is PageRank vector to send the pagerank of y to data center 1.

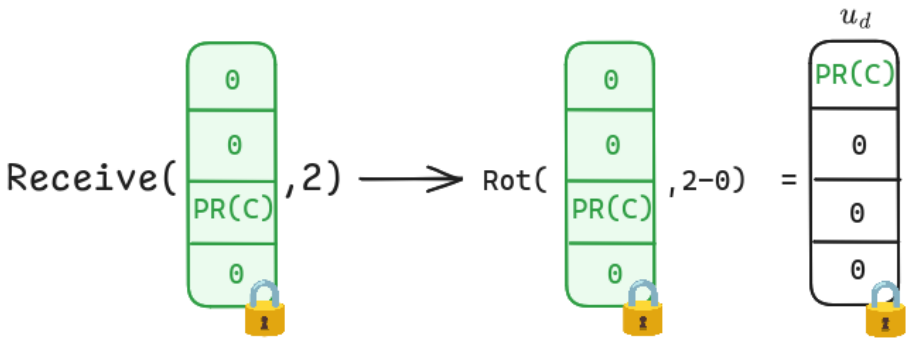


Computation step

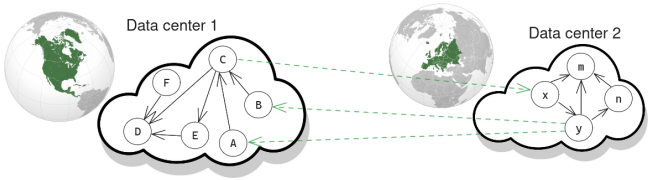


For each iteration:

- Synchronize with connected data centers: creation of an additionnal vector u_d.



Computation step

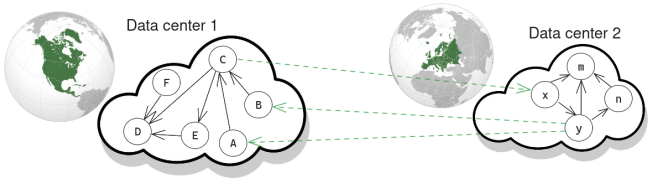


For each iteration:

- Synchronize with connected data centers: creation of an additionnal vector u_d.
- Update the local pagerank: computing the formula $\frac{1-d}{k_{max}} + d \cdot (\sum u_j + u_d)$.

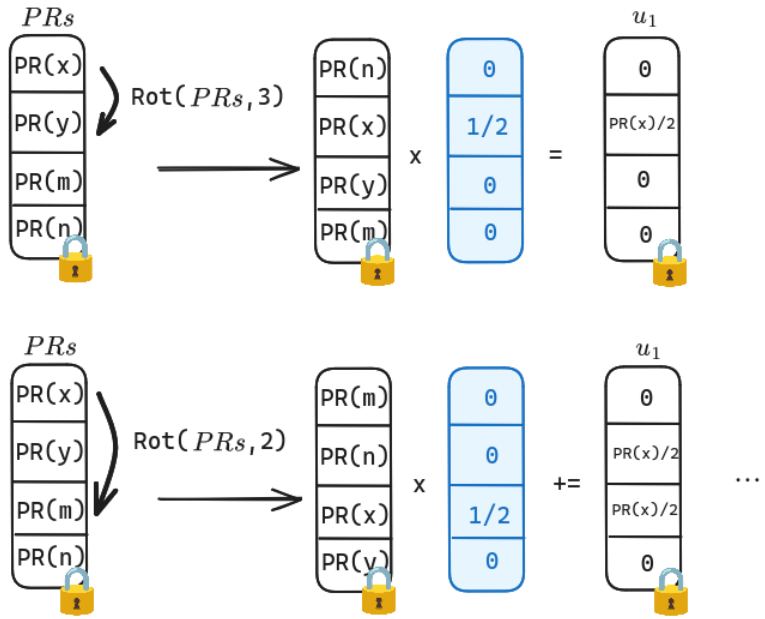
$$PRs = \begin{bmatrix} PR(x) \\ PR(y) \\ PR(m) \\ PR(n) \end{bmatrix} = \begin{bmatrix} \frac{1-d}{3} \\ \frac{1-d}{3} \\ \frac{1-d}{3} \\ \frac{1-d}{3} \end{bmatrix} + \begin{bmatrix} d \\ d \\ d \\ d \end{bmatrix} \times \left\langle \begin{matrix} u_1 \\ \begin{bmatrix} 0 \\ PR(x)/2 \\ PR(x)/2 \\ PR(y)/4 \end{bmatrix} \end{matrix} + \begin{matrix} u_2 \\ \begin{bmatrix} 0 \\ 0 \\ PR(y)/4 \\ 0 \end{bmatrix} \end{matrix} + \begin{matrix} u_3 \\ \begin{bmatrix} 0 \\ 0 \\ PR(n) \\ 0 \end{bmatrix} \end{matrix} + \begin{matrix} u_d \\ \begin{bmatrix} PR(C) \\ 0 \\ 0 \\ 0 \end{bmatrix} \end{matrix} \right\rangle$$

Computation step

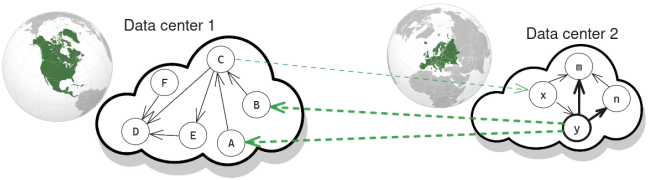


For each iteration:

- Synchronize with connected data centers: creation of an additionnal vector u_d.
- Update the local pagerank: computing the formula $\frac{1-d}{k_{max}} + d \cdot (\sum u_j + u_d)$.
- Preparation for the next iteration (similar to *pre-processing step*).

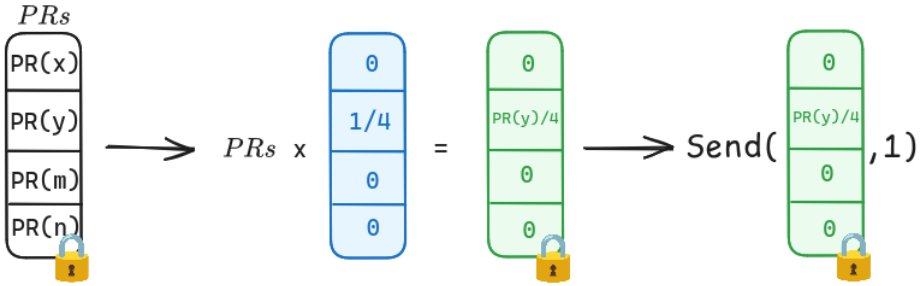


Computation step

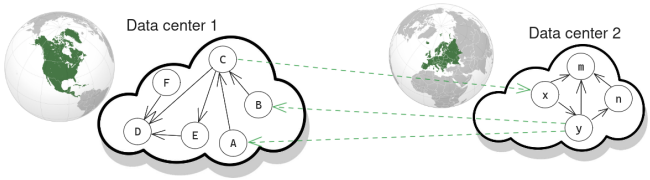


For each iteration:

- Synchronize with connected data centers: creation of an additionnal vector u_d.
- Update the local pagerank: computing the formula $\frac{1-d}{k_{max}} + d \cdot (\sum u_j + u_d)$.
- Preparation for the next iteration (similar to *pre-processing step*).



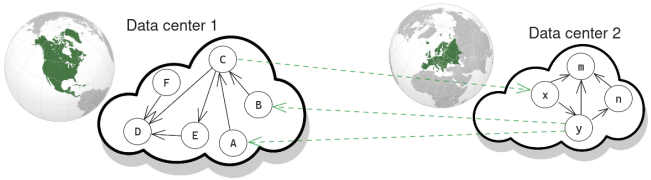
Computation step



Last iteration:

- Synchronize with connected data centers: additionnal vector u_d for the **active nodes** and an other for the **dangling nodes**.
- Update the local pagerank: computing the formula $\frac{1-d}{k_{max}} + d \cdot (\sum u_j + u_d)$ for the **active nodes** and for the **dangling nodes**.

Computation step



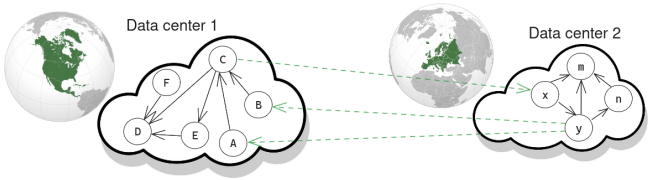
Last iteration:

- Synchronize with connected data centers: additionnal vector u_d for the **active nodes** and an other for the **dangling nodes**.
- Update the local pagerank: computing the formula $\frac{1-d}{k_{max}} + d \cdot (\sum u_j + u_d)$ for the **active nodes** and for the **dangling nodes**.

After the last iteration, each data center has:

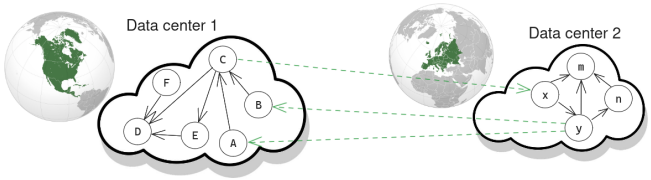
- PRs: ciphertext with pagerank values of active nodes,
- $\overline{PRs}$: ciphertext with pagerank values of dangling nodes.

Completion step



- The data center first receives the result of its decrypted vectors (PRs and $\overline{PRs}$) from the trusted entity.

Completion step

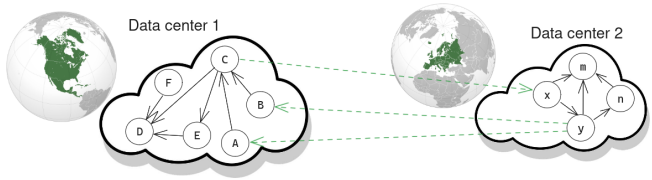


- The data center first receives the result of its decrypted vectors (PRs and $\overline{PRs}$) from the trusted entity.
- Then, using a mapping from a node to a rule, the data center recovers the pagerank values of all his nodes.

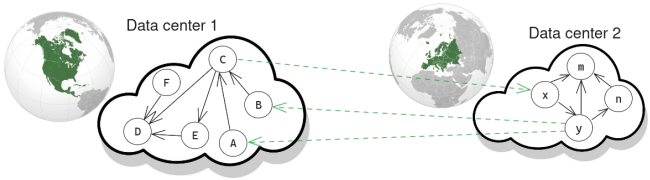
x $\leftarrow$ active node
y $\leftarrow$ active node
m $\leftarrow$ dangling node
n $\leftarrow$ active node

POPPY$_M$ and POPPY$_{M2}$

- Also use the SIMD capabilities of CKKS.

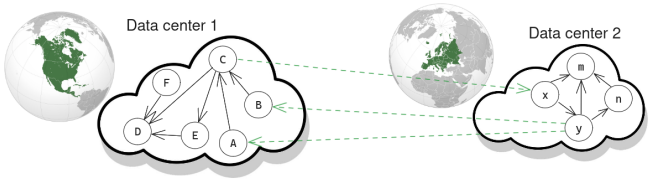


POPPY$_M$ and POPPY$_{M2}$



- Also use the SIMD capabilities of CKKS.
- **Matrix representation** to avoid the costly reconstruction of the set of u_j's.

POPPY$_M$ and POPPY$_{M2}$



- Also use the SIMD capabilities of CKKS.
- **Matrix representation** to avoid the costly reconstruction of the set of u_j's.
- The pruning rules are compatible with POPPY$_M$ and POPPY$_{M2}$.

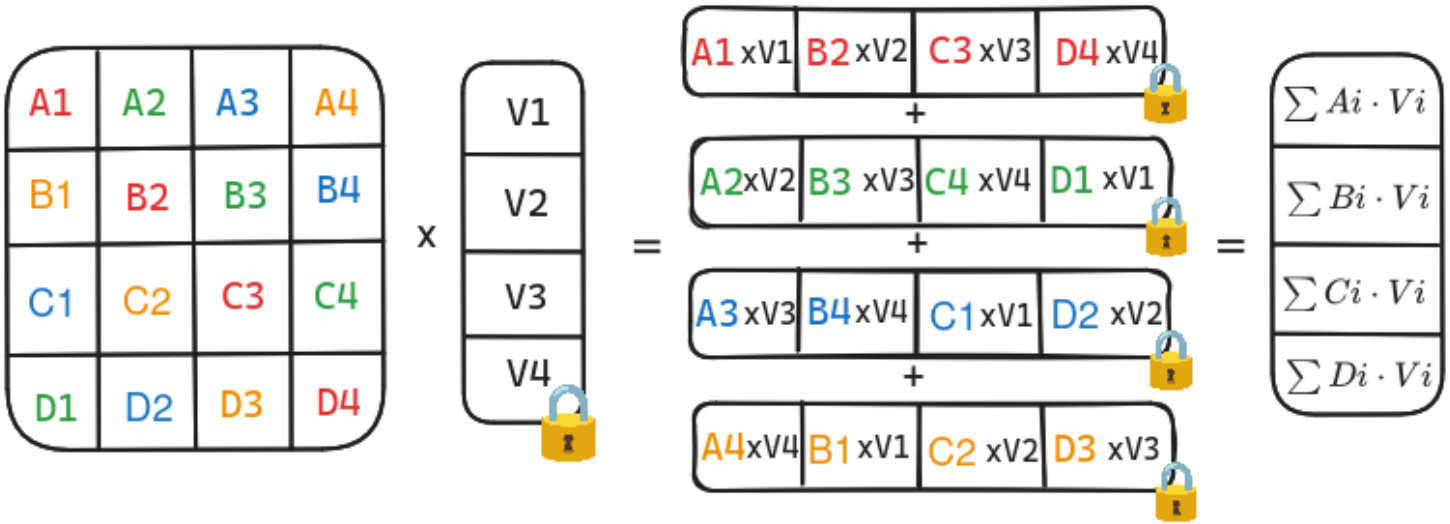
$$PRs = \begin{pmatrix} PR(x) \\ PR(y) \\ PR(m) \\ PR(n) \\ 0 \end{pmatrix} = \begin{pmatrix} \frac{1-d}{3} \\ \frac{1-d}{3} \\ \frac{1-d}{3} \\ \frac{1-d}{3} \\ \frac{1-d}{3} \end{pmatrix} + \begin{pmatrix} d \\ d \\ d \\ d \\ d \end{pmatrix} \times \begin{pmatrix} 0 & 0 & 0 & 0 & 1 \\ \frac{1}{2} & 0 & 0 & 0 & 0 \\ \frac{1}{2} & \frac{1}{4} & 0 & 1 & 0 \\ 0 & \frac{1}{4} & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix} \times \begin{pmatrix} PR(x) \\ PR(y) \\ PR(m) \\ PR(n) \\ PR(c) \end{pmatrix}$$

$$PRs = \begin{pmatrix} PR(x) \\ PR(y) \\ PR(m) \\ PR(n) \end{pmatrix} = \begin{pmatrix} \frac{1-d}{3} \\ \frac{1-d}{3} \\ \frac{1-d}{3} \\ \frac{1-d}{3} \end{pmatrix} + \begin{pmatrix} d \\ d \\ d \\ d \end{pmatrix} \times \begin{pmatrix} 0 & 0 & 0 & 0 \\ \frac{1}{2} & 0 & 0 & 0 \\ \frac{1}{2} & \frac{1}{4} & 0 & 1 \\ 0 & \frac{1}{4} & 0 & 0 \end{pmatrix} \times \begin{pmatrix} PR(x) \\ PR(y) \\ PR(m) \\ PR(n) \end{pmatrix} + \overbrace{\begin{pmatrix} PR(C) \\ 0 \\ 0 \\ 0 \end{pmatrix}}^{u_d}$$

Secure matrix multiplication

Matrix multiplication using Halevi-Shoup algorithm [7]:

- Each generalized diagonal is multiplied with the i-th rotated PRs vector

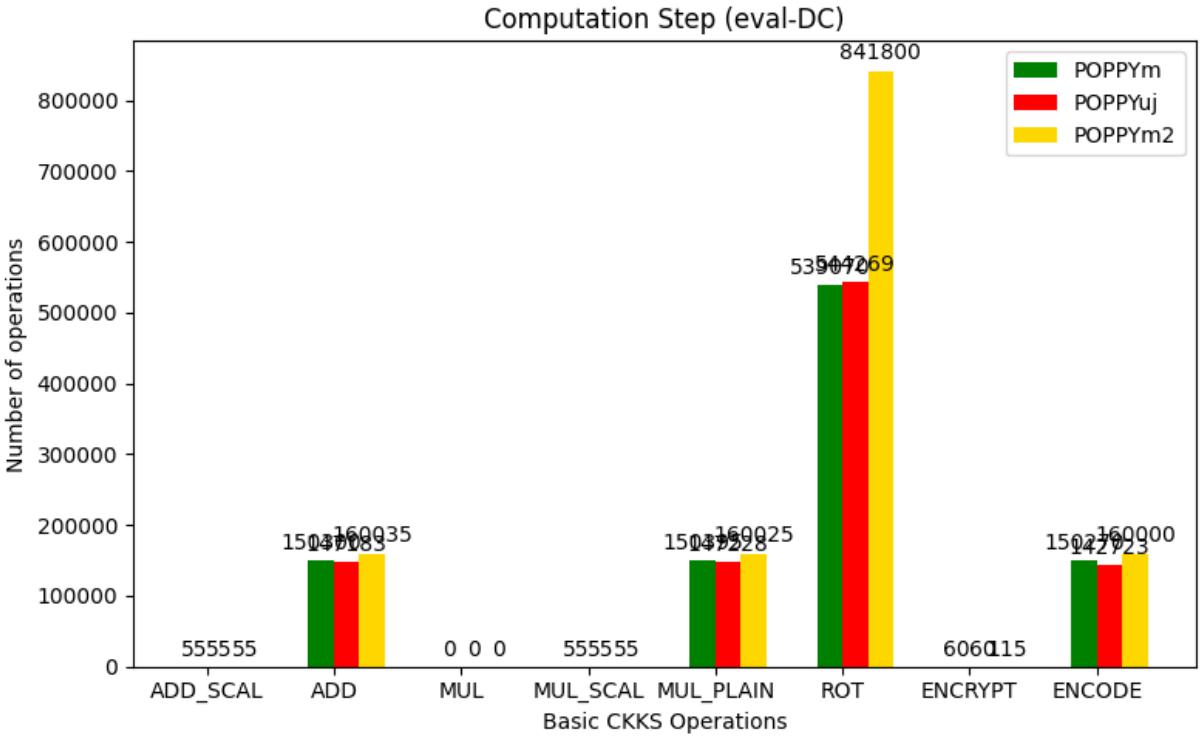


- An optimization from [1] is used to save memory: we generated only **log(n)** rotation keys instead of n: composition of rotation.

Progress of the Talk

1. Introduction
2. Preliminaries
3. POPPY
4. Experimental results
5. Conclusion

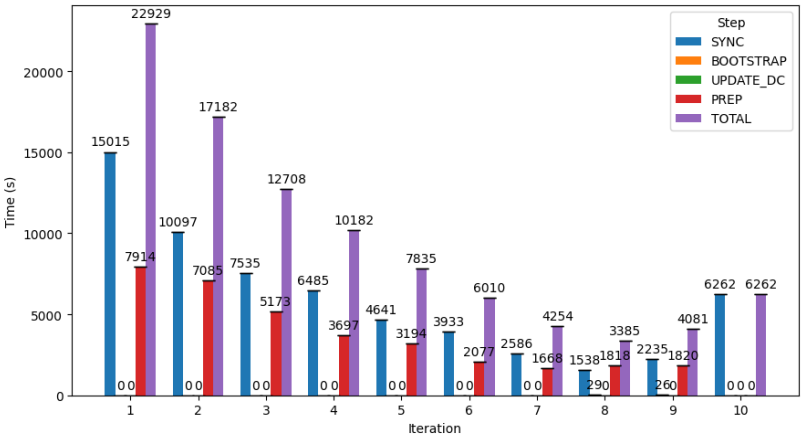
Experimental results – Performance (1/2)



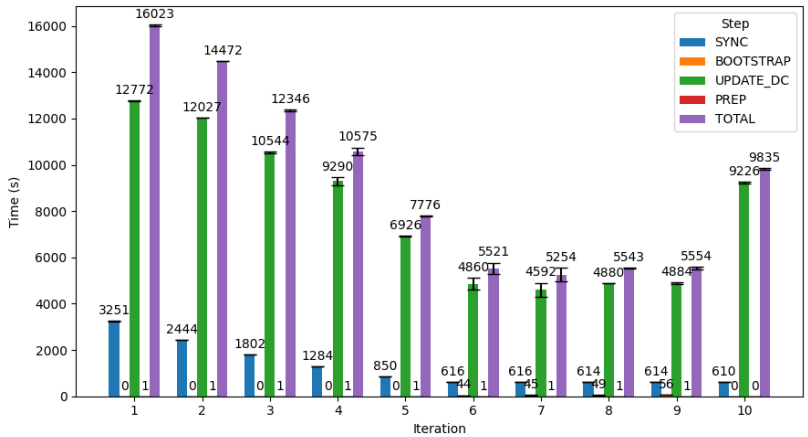
Real graph `P2P-Gnutella04` with 10 876 nodes and divided in 5 data centers (Twitter partition)
Ring dimension: 131072
Multiplicative depth: 25
Specifications: `roazhon3` (2018)

- 2 CPUs Intel Xeon Gold 6142
- 16 cores/CPU
- 1TB RAM
- 1.6TB HDD

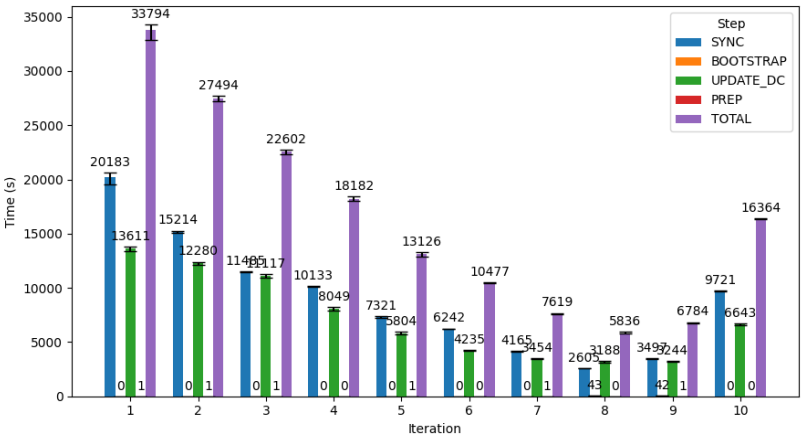
Experimental results – Performance (2/2)



POPPY$_{u_j}$ with pruning



POPPY$_{M}$ with pruning

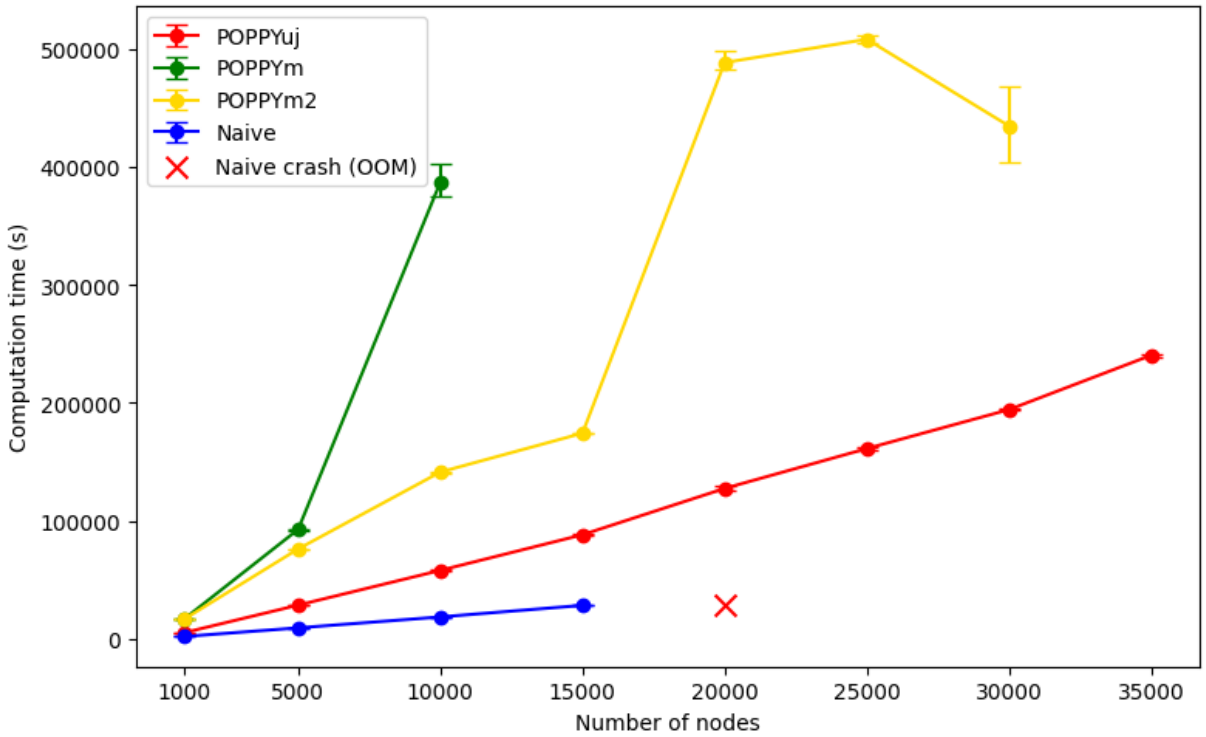


POPPY$_{M2}$ with pruning

Real graph `P2P-Gnutella04` with 10 876 nodes and divided in 5 data centers (Twitter partition)
Ring dimension: 131072
Multiplicative depth: 25
Specifications: `roazhon3` (2018)

- 2 CPUs Intel Xeon Gold 6142
- 16 cores/CPU
- 1TB RAM
- 1.6TB HDD

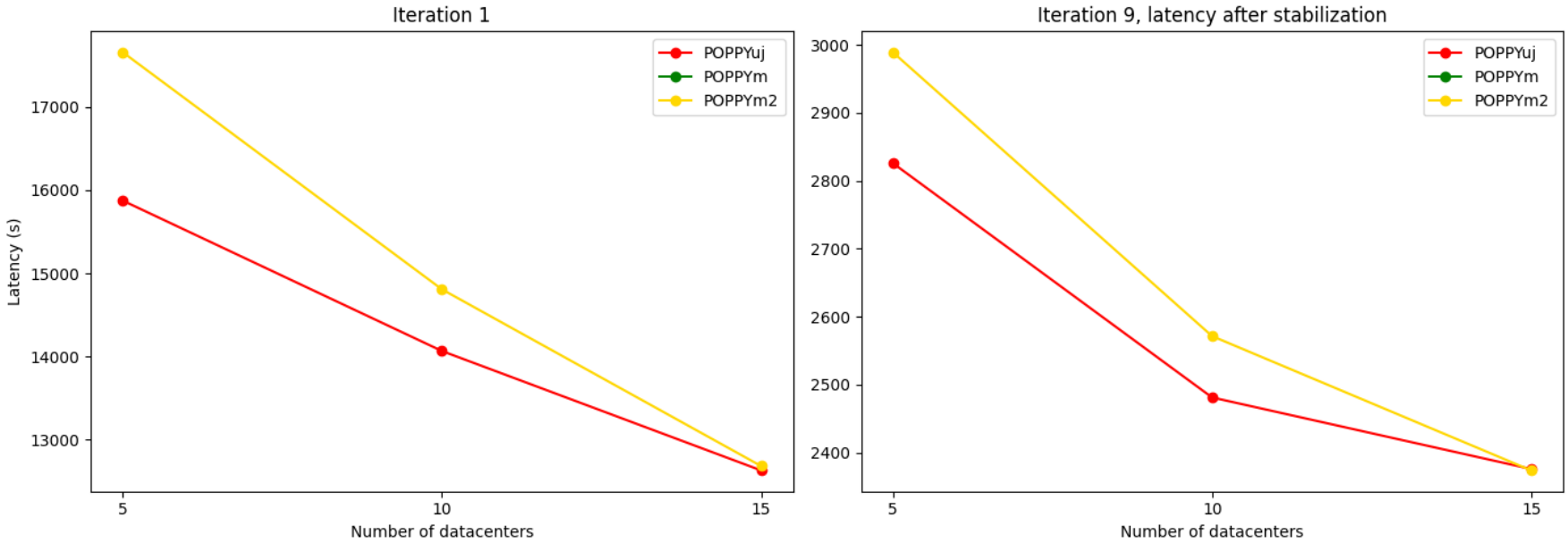
Experimental results – Scalalility (1/2)



Synthetic graphs with 5 data centers (Twitter partition) and increasing number of nodes
Ring dimension: 131072
Multiplicative depth: 25
Specifications: `roazhon9` (2017)

- 2 CPUs Intel Xeon Gold 5118
- 12 cores/CPU
- 512GB RAM
- 1.2TB HDD

Experimental results – Scalalility (2/2)



Synthetic graphs with 25 000 nodes and increasing number of data centers (uniform partition)
Ring dimension: 131072
Multiplicative depth: 25
Specifications: `roazhon9` (2017)

- 2 CPUs Intel Xeon Gold 5118
- 12 cores/CPU
- 512GB RAM
- 1.2TB HDD

Progress of the Talk

1. Introduction
2. Preliminaries
3. POPPY
4. Experimental results
5. Conclusion

Conclusion

Graph computing is important nowadays and preserving the privacy of personal data is a major issue.

Conclusion

Graph computing is important nowadays and preserving the privacy of personal data is a major issue.

Three POPPY algorithms with different trade-offs

The baseline: follows the "Think Like a Vertex" paradigm.
POPPY$_{u_j}$: vector representation using SIMD capabilities of CKKS.
POPPY$_M$ and POPPY$_{M2}$: matrix representation.

Conclusion

Graph computing is important nowadays and preserving the privacy of personal data is a major issue.

Three POPPY algorithms with different trade-offs

The baseline: follows the "Think Like a Vertex" paradigm.
POPPY$_{u_j}$: vector representation using SIMD capabilities of CKKS.
POPPY$_M$ and POPPY$_{M2}$: matrix representation.

Future work

Allow each data center to choose the best POPPY algorithm according to its subgraph (based on the cost model).
Dealing with covert data centers.
Adaptation of POPPY to HITS (polynomial interpolation is required).

Thank you!

[1] I. Ahmad, L. Sarker, D. Agrawal, A. E. Abbadi, and T. Gupta.
Coeus: A system for oblivious document ranking and retrieval.
Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles, 2021.

[2] S. Azogagh, Z. A. Birba, S. Gambs, and M.-O. Killijian.
Crypto'graph: Leveraging privacy-preserving distributed link prediction for robust graph learning.
ArXiv, abs/2309.10890, 2023.

[3] F. Boemer, Y. Lao, R. Cammarota, and C. Wierzynski.
ngraph-he: a graph compiler for deep learning on homomorphically encrypted data.
In *Proceedings of the 16th ACM International Conference on Computing Frontiers*, CF '19, page 3–13, New York, NY, USA, 2019. Association for Computing Machinery.

[4] J. H. Cheon, A. Kim, M. Kim, and Y. Song.
Homomorphic encryption for arithmetic of approximate numbers.
In *International Conference on the Theory and Application of Cryptology and Information Security*, 2017.

[5] I. Damgård and M. Jurik.
A generalisation, a simplification and some applications of paillier's probabilistic public-key system.
In *Public Key Cryptography, 4th International Workshop on Practice and Theory in Public Key Cryptography*, volume 1992, pages 119–136. Springer, 2001.

[6] W. Diffie and M. E. Hellman.
New directions in cryptography.
Democratizing Cryptography, 1976.

[7] S. Halevi and V. Shoup.
Algorithms in helib.
Cryptology ePrint Archive, Paper 2014/106, 2014.

[8] W. Hu, X. Xia, X. Ding, X. Zhang, K. Zhong, and H. Zhang.
Smpc-ranking: A privacy-preserving method on identifying influential nodes in multiple private networks.
IEEE Transactions on Systems, Man, and Cybernetics: Systems, 53:2971–2982, 2023.

[9] I. C. F. Ipsen and T. M. Selee.
Pagerank computation, with special attention to dangling nodes.
SIAM Journal on Matrix Analysis and Applications, 29(4):1281–1296, 2008.

[10] J. Kim, D. Koo, Y. Kim, H. Yoon, J. Shin, and S. Kim.
Efficient privacy-preserving matrix factorization for recommendation via fully homomorphic encryption.
ACM Transactions on Privacy and Security (TOPS), 21:1 – 30, 2018.

[11] V. B. Kukkala.
Privacy preserving distributed analysis of social networks.

In *Companion Proceedings of the The Web Conference 2018*, WWW '18, page 873–877, Republic and Canton of Geneva, CHE, 2018. International World Wide Web Conferences Steering Committee.

[12] V. B. Kukkala and S. Iyengar.
Computing betweenness centrality: An efficient privacy-preserving approach.
pages 23–42, 2018.

[13] V. B. Kukkala and S. S. Iyengar.
Identifying influential spreaders in a social network (while preserving privacy).
Proceedings on Privacy Enhancing Technologies, 2020:537 – 557, 2020.

[14] L. Liang, J. Lin, Z. Qu, I. Ahmad, F. Tu, T. Gupta, Y. Ding, and Y. Xie.
Spg: Structure-private graph database via squeezepir.
Proc. VLDB Endow., 16:1615–1628, 2023.

[15] G. Malewicz, M. H. Austern, A. J. C. Bik, J. C. Dehnert, I. Horn, N. Leiser, and G. Czajkowski.
Pregel: a system for large-scale graph processing.
Proceedings of the 2010 ACM SIGMOD International Conference on Management of data, 2010.

[16] B. D. McKay and A. Piperno.
Practical graph isomorphism, ii.
Journal of Symbolic Computation, 60:94–112, 2014.

[17] A. Sangers, M. van Heesch, T. Attema, T. Veugen, M. Wiggerman, J. Veldsink, O. Bloemen, and D. T. H. Worm.

Secure multiparty pagerank algorithm for collaborative fraud detection.
In *IACR Cryptology ePrint Archive*, 2019.

[18] A. C. Zhou, R. Qiu, T. Lambert, T. Allard, S. Ibrahim, and A. El Abbadi.
PGPregel: An End-to-End System for Privacy-Preserving Graph Processing in Geo-Distributed Data Centers.
In *SoCC '22: ACM Symposium on Cloud Computing*, pages 386–402, San Francisco California, United States, 2022. Association for Computing Machinery.

Cost model of POPPY

BatchSize_uj = NextPowerTwo(ActiveNodes)
BatchSize_m = NextPowerTwo(ActiveNodes + DistanNodes)

Preparation	Rotation	Addition	Multiplication
$POPPY_{u_j}$	len(E_dist)	len(E_dist)	len(E_dist)
$POPPY_{M}$	len(V_dist)	len(V_dist)	len(V_dist)
$POPPY_{M2}$	len(E_dist)	len(E_dist)	len(E_dist)

Evaluation	Rotation	Addition	Multiplication
$POPPY_{u_j}$	0	0	0
$POPPY_{M}$	BatchSize_m	BatchSize_m	BatchSize_m
$POPPY_{M2}$	BatchSize_uj	BatchSize_uj	BatchSize_uj

Reconstruction	Rotation	Addition	Multiplication
$POPPY_{u_j}$	len(E_loc)	len(E_loc)	len(E_loc)+4
$POPPY_{M}$	0	0	4
$POPPY_{M2}$	0	0	4